NumPy(Numerical Python)

• NumPy (Numerical Python) is a fundamental library for numerical computing in Python. • It is a Python library for creating N dimensional arrays(1D,2D,3D) • It also provide mathematical functions, and linear algebra operations, statistical distribution ,trigonometric and random number capability making it essential for data science, machine learning, and scientific computing. • Ability to quickly broadcast function, Broadcasting allows NumPy to perform operations between arrays of different shapes without explicit looping. It automatically expands smaller arrays to match the shape of larger ones, making operations efficient. • NumPy arrays and Python lists both store collections of data, but NumPy arrays are faster, more memory-efficient, and optimized for numerical operations.

```
# create numpy array
```

```
#import numpy as np
```

```
#A NumPy array looks similar to a Python list, but it is specially designed for numerical operations.
```

```
#Creating N-Dimensional Arrays in NumPy
```

```
import  numpy as  np
arr = [10, 20, 30, 40, 50,70]
print(arr)
print(type(arr))
```

```
[10, 20, 30, 40, 50, 70]
<class 'list'>
```

```
#one dimensional
arr1=np.array(arr)
print(arr1)
type(arr1)
```

```
[10 20 30 40 50 70]
numpy.ndarray
```

```
arr51=np.array([10,20,30,45])
print(arr51)
```

```
[10 20 30 45]
```

```
#2-D Array
```

```
#Converting a 2D List (Nested List) to a NumPy Array
```

```
a = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
print(a)
```

```
[[10 20 30]
 [40 50 60]]
```

```
#creating direct 2d array
```

```
marks = np.array([
    [80, 75, 90],
    [85, 70, 95],
    [60, 65, 72]
])

print(marks)
```

```
[[80 75 90]
 [85 70 95]
 [60 65 72]]
```

```
#3-D Array , 3D array contains multiple 2D arrays.
```

```
a = np.array([
    [
        [10, 20, 30],
        [40, 50, 60]
    ],
    [
        [70, 80, 90],
        [100, 110, 120]
    ],
   [ [1,3,4],
    [10,12,78]]

])


print(a)
```

```
[[[ 10  20  30]
  [ 40  50  60]]

 [[ 70  80  90]
  [100 110 120]]

 [[  1   3   4]
  [ 10  12  78]]]
```

np.zeros() – Create NumPy Arrays Filled with Zeros • The np.zeros() function creates an array filled with zero values, useful for initializing arrays in numerical computing, data science, and machine learning. • Create the shape.

Syntax import numpy as np np.zeros(shape, dtype=float)

• shape ⟶ The shape of the array (integer for 1D, tuple for multi-dimensional). • dtype ⟶ (Optional) The data type of the array elements (default = float).

When to Use np.zeros() • Initializing arrays before computation (avoids uninitialized memory issues). • Creating placeholders for machine learning models & deep learning weights. • Allocating memory for matrices before filling them with actual values.

```
#creates an array filled with zero values,
a=np.zeros((5,3),dtype=int)
print(a)
```

```
[[0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
```

```
#Creating a 1D Zero Array
arr = np.zeros(5,dtype=int)
print(arr)
```

```
[0 0 0 0 0]
```

```
a = np.zeros((2, 3), dtype=int)
print(a)
```

```
[[0 0 0]
 [0 0 0]]
```

```
#create a 3D zero matrix (3D array) np.zeros((layers, row, column), datatype)
a3d = np.zeros((3, 4, 4), dtype=int)
print(a3d)
```

```
[[[0 0 0 0]
  [0 0 0 0]
  [0 0 0 0]
  [0 0 0 0]]

 [[0 0 0 0]
  [0 0 0 0]
  [0 0 0 0]
  [0 0 0 0]]
```

```
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]]
```

np.ones() – Create NumPy Arrays Filled with Ones

The np.ones() function creates an array filled with ones, useful for initializing arrays in numerical computing, data science, and machine learning.

Syntax

import numpy as np np.ones(shape, dtype=float)

```
#Create NumPy Arrays Filled with Ones 1D
arr2=np.ones(5)
print(arr2)
```

```
[1. 1. 1. 1. 1.]
```

```
#Create NumPy Arrays Filled with Ones 2D
arr2=np.ones((2,3))
print(arr2)
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

```
#Create NumPy Arrays Filled with Ones 3D
arr3=np.ones((4,3,4))
print(arr3)
```

```
[[[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]
```

```
#Create identity matrix
arr2=np.identity(5)
print(arr2)
```

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

np.eye() : Create an Identity Matrix in NumPy • The np.eye() function creates an identity matrix, a square matrix with 1s on the diagonal and 0s everywhere else. • Identity matrices are widely used in linear algebra, machine learning, and deep learning. Syntax import numpy as np np.eye(N, M=None, k=0, dtype=float)

• N ⟶ Number of rows. • M ⟶ (Optional) Number of columns (default = N, creating a square matrix). • k ⟶ (Optional) Diagonal offset (0 for main diagonal, positive for upper diagonals, negative for lower diagonals). • dtype ⟶ (Optional) Data type of the output matrix (default = float).

```
#2D identity matrix
arr9 = np.eye(4)
print(arr9)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
```

```
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
#3d identity matrix
arr5 = np.tile(np.eye(3,3), (3,1,1))
print(arr5)
```

```
[[[1. 0. 0.]
  [0. 1. 0.]
  [0. 0. 1.]]

 [[1. 0. 0.]
  [0. 1. 0.]
  [0. 0. 1.]]

 [[1. 0. 0.]
  [0. 1. 0.]
  [0. 0. 1.]]]
```

np.arange() – Creating NumPy Arrays with Ranges • The np.arange() function in NumPy creates an array with evenly spaced values between a start and stop value. • It works similarly to Python's range() but produces a NumPy array instead of a list.

Syntax

Import numpy as np np.arange(start, stop, step, dtype)

• start :(Optional) Starting value (default = 0) • stop :The end value (exclusive) • step :(Optional) Step size (default = 1) • dtype :(Optional) Data type of the array

```
#generate array of given range
arr6 = np.arange(25)
print(arr6)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24]
```

```
arr7 = np.arange(1, 25)
print(arr7)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]
```

```
arr7 = np.arange(1,25,2)  # Start from 0, stop before 10
print(arr7)
```

```
[ 1  3  5  7  9 11 13 15 17 19 21 23]
```

np.linspace() – Create Evenly Spaced Numbers in NumPy import numpy as np Syntax np.linspace(start, stop, num=50, endpoint=True, dtype=None) • start ⟶ The starting value of the sequence. • stop ⟶ The ending value of the sequence. • num ⟶ (Optional) Number of values to generate (default = 50). • endpoint ⟶ (Optional) If True (default), stop is included. If False, stop is excluded. • dtype ⟶ (Optional) Data type of the output array

```
#Create Evenly Spaced Numbers
arr8=np.linspace(0, 100,5,dtype=int)
print(arr8)
```

```
[  0  25  50  75 100]
```

```
#copy any array
arr9=arr8.copy()
print(arr9)
```

```
[  0  25  50  75 100]
```

Numpy property and attributes

Shape ⟶ arrangement

ndim ⟶ dimensions

size → total elements

itemsize → memory per element

dtype → data type

astype() → convert data type

shape : how the array is arranged in each dimension.

```
print(arr1)
print(arr1.shape)
```

```
[10 20 30 40 50 70]
(6,)
```

```
print(arr2)
print(arr2.shape)
```

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
(5, 5)
```

```
print(a)
print(a.shape)
```

```
[[0 0 0]
 [0 0 0]]
(2, 3)
```

```
narr=np.array([
    [
        [10, 20],
        [40, 50]
    ],
    [
        [70, 80],
        [100, 110]
    ],
   [ [1,3],
    [10,12]]
])
print(narr)
print(narr.shape)
```

```
[[[ 10  20]
  [ 40  50]]

 [[ 70  80]
  [100 110]]

 [[  1   3]
  [ 10  12]]]
(3, 2, 2)
```

ndim

ndim is an attribute returns the number of dimensions (axes) of the array.

```
print(arr1)
print(arr1.ndim)
```

```
[10 20 30 40 50 70]
1
```

```
print(arr3)
print("dimension is",arr3.ndim)
```

```
[[[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]
dimension is 3
```

size

size returns the total number of elements contained in the array.

```
print(arr1)
print(arr1.size)
```

```
[10 20 30 40 50 70]
6
```

```
print(marks)
print(marks.size)
```

```
[[80 75 90]
 [85 70 95]
 [60 65 72]]
9
```

```
print(a)
print(a.size)
```

```
[[0 0 0]
 [0 0 0]]
6
```

itemsize

itemsize returns the number of bytes occupied by a single element of the array.

```
print(arr1)
print(arr1.itemsize,"bytes")
```

```
[10 20 30 40 50 70]
8 bytes
```

```
print(marks)
print(marks.itemsize,"bytes")
```

```
[[80 75 90]
 [85 70 95]
 [60 65 72]]
8 bytes
```

```
print(a)
print(a.itemsize,"bytes")
```

```
[[0 0 0]
 [0 0 0]]
8 bytes
```

- dtype :It tells us the data type of the values.

```
print(arr1)
print(arr1.dtype)
```

```
[10 20 30 40 50 70]
int64
```

```
import numpy as np
arr10=np.arange(4,25,3,dtype=int)
print(arr10)
print(arr10.dtype)
```

```
[ 4  7 10 13 16 19 22]
int64
```

- astype() :It is used when we want to convert one data type into another.

```
new_arr = arr1.astype(float)
print(new_arr)
print(arr1.dtype)
print(new_arr.dtype)
```

```
[10. 20. 30. 40. 50. 70.]
int64
float64
```

Array Reshaping and Flattening

Reshape Array: changing the dimensions (e.g., rows and columns) without altering the actual data

Flatten Array: transforming a multi-dimensional array into a single-dimensional array

- convert one dimension into two

```
arr11 = np.array([1,2,3,4,5,6])
print("one dimensional array",arr11)
print("two dimensional array")
print(arr11.reshape(2,3))
```

```
one dimensional array [1 2 3 4 5 6]
two dimensional array
[[1 2 3]
 [4 5 6]]
```

- convert two dimension into one

```
print(arr11.flatten())
```

```
[1 2 3 4 5 6]
```

Array Stacking and Splitting

- Array Stacking: combining multiple arrays along a new axis, resulting in an array with a higher dimension than the input arrays

vertical stacking - row wise -> output array is 2D

horizontal stacking - column wise -> output array is 1D

```
arr12 = np.array([1,2,3,4,5])
arr13 = np.array([4,5,6,7,8])
print(np.vstack((arr12,arr13)))
print(np.hstack((arr12,arr13)))
```

```
[[1 2 3 4 5]
 [4 5 6 7 8]]
[1 2 3 4 5 4 5 6 7 8]
```

splitting

```
arr14 = np.array([[1,2,3,23], [4,5,6,32]])
print(arr14)
```

```
[[ 1  2  3 23]
 [ 4  5  6 32]]
```

```
hsplit = np.hsplit(arr14,4)

for s in hsplit:
    print(s)
```

```
[[1]
 [4]]
[[2]
 [5]]
[[3]
 [6]]
[[23]
 [32]]
```

```
vsplit = np.vsplit(arr14,2) # vertical split -> input and output array are same dimention
for s in vsplit:
    print(s)
```

```
[[ 1  2  3 23]]
[[ 4  5  6 32]]
```

Mathematical Operations on Array

```
arr15=np.array([10,20,30,40,50])
print(arr15+10)
print(arr15-10)
print(arr15*2)
print(arr15/10)
print(arr15//10)
```

```
[20 30 40 50 60]
[ 0 10 20 30 40]
[ 20  40  60  80 100]
[1. 2. 3. 4. 5.]
[1 2 3 4 5]
```

```
arr16=np.array([2,4,6])
print(np.square(arr16)) # square of all array elements
print(np.sqrt(arr16),"")
```

```
[ 4 16 36]
[1.41421356 2.         2.44948974]
```

Mathematical Operations on Muliple Arrays

```
arr17 = np.array([1,2,3])
arr18 = np.array([4,5,6])
print(arr17)
print(arr18)
```

```
[1 2 3]
[4 5 6]
```

```
print(np.add(arr17,arr18))
print(np.subtract(arr17,arr18))
print(np.multiply(arr17,arr18))
print(np.divide(arr17,arr18))
```

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

```
# Dot product: computes the inner product of the two vectors and sum the results
print(np.dot(arr17,arr18)) # dot product
```

```
32
```

Transpose: converting rows into columns and columns into rows.

```
print(arr17.T)
```

```
[1 2 3]
```

```
t = np.array([[1,2,3],
              [4,5,6]])

print(t)    # initial shape: 2,3
print(t.T)  # shape after transpose: 3,2
```

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

Statistical Operations

Double-click (or enter) to edit

```
marks = np.array([80, 75, 90, 65, 85,75])
print("mean",np.mean(marks))
print("median",np.median(marks))
values, counts = np.unique(marks, return_counts=True)
mode = values[np.argmax(counts)]
print("mode",mode)
print("min",np.min(marks))
print("max",np.max(marks))
print("total",np.sum(marks))
print("Standard Deviation",np.std(marks))
```

```
mean 78.33333333333333
median 77.5
mode 75
min 65
max 90
total 470
Standard Deviation 7.993052538854533
```

A college wants to analyze the marks of 10 students in 3 subjects. The marks are stored in a NumPy array:

Write a Python program using NumPy to:

Display the complete array.

Find the shape, ndim, size, and dtype.

Find the total marks.

Find the highest and lowest marks.

Calculate the average marks of all students.

A shop records the sales of 3 products for 4 days. The data is represented using a NumPy array:

Write a Python program to:

Display the sales array.

Display its shape, ndim, size, and dtype.

Calculate the total sales.

Find the average sales.

Find the maximum and minimum sales.

Weather Data Analysis Using a 3D Array

A weather monitoring system records temperature for 2 cities, 3 days, and 4 readings per day. Create a suitable 3D NumPy array containing temperature values.

Write a Python program to:

Create and display the 3D array.

Display its shape, ndim, size, and dtype.

Find the maximum temperature.

Find the minimum temperature.

Calculate the average temperature.

Calculate the total of all temperature readings.

```
#Indexing is the process of accessing a particular element of an array using its position (index).
```

```
marks = np.array([80, 75, 90, 65, 85,45,78,23,89])
print(marks[4])
print(marks[7])
```

```
85
23
```

```
import numpy as np
```

```
arr11=np.arange(24).reshape(8, 3)
arr11
print(arr11)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]
 [15 16 17]
 [18 19 20]
 [21 22 23]]
```

```
arr11=np.arange(24).reshape(6, 4)
arr11
print(arr11)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
```

```
#indexing 2D array
print(arr11[2, 2])
```

```
10
```

```
#indexing 3D array

arr12 = np.array([
    [
        [10, 20, 30],
        [40, 50, 60]
    ],
    [
        [70, 80, 90],
        [100, 110, 120]
    ],
   [ [12,23,11],
    [44,33,22]]

])
print(arr12)
print(arr12.shape)
```

```
[[[ 10  20  30]
  [ 40  50  60]]

 [[ 70  80  90]
  [100 110 120]]

 [[ 12  23  11]
  [ 44  33  22]]]
(3, 2, 3)
```

```
print(arr12[1, 1, 1])
```

```
110
```

```
print(arr12[2, 0, 1])
```

```
23
```

Slicing in NumPy

Slicing means extracting a part or a range of elements from an array.

Unlike indexing, which usually selects one element, slicing selects multiple elements.

array[start : stop : step]

start ⟶ where slicing begins

stop ⟶ where slicing ends; this index is not included

step ⟶ how many positions to move at a time

```
#Slicing in a 1D Array
print(arr1)
print(arr1[1:3])
print(arr1[:3])
print(arr1[3:])
print(arr[::2])
```

```
[10 20 30 40 50 70]
[20 30]
[10 20 30]
[40 50 70]
[0 0 0]
```

```
arr11[3]
```

```
array([12, 13, 14, 15])
```

```
arr11[:,3]
```

```
array([ 3,  7, 11, 15, 19, 23])
```

Slicing in a 2D Array

```
#Slicing
```

```
marks = np.array([80, 75, 90, 65, 85])
print(marks[1:4])
print(marks[:2])
print(marks[2:])
```

```
[75 90 65]
[80 75]
[90 65 85]
```

```
import numpy as np
a = np.array([
    [
        [10, 20, 30],
        [40, 50, 60]
    ],
    [
        [70, 80, 90],
        [100, 110, 120]
    ],
    [
        [50, 70, 90],
        [23, 11, 12]
    ]
```

```
])
print(a[1:4])
print(a[:2])
print(a[2:])
```

```
[[[ 70  80  90]
  [100 110 120]]

 [[ 50  70  90]
  [ 23  11  12]]]
[[[ 10  20  30]
  [ 40  50  60]]

 [[ 70  80  90]
  [100 110 120]]]
[[[50 70 90]
  [23 11 12]]]
```

```
marks = np.array([40, 55, 70, 85, 90])
print(marks[marks > 60])
```

```
[70 85 90]
```

```
#Arithmetic Operations
```

```
Start coding or generate with AI.
```

```
arr = np.arange(1, 10, 3)  # Start=1, Stop=10, Step=2
print(arr)
```

```
[1 4 7]
```